Assignment 2 (Weeks 3 and 4)

Instructions

- This assignment covers material from Weeks 3 and 4.
- Submit a single PDF file containing all parts of the assignment.
- For SQL questions, write queries as plain text (not screenshots). Indicate the dialect-neutral standard SQL syntax, you do not need to run the queries, but your syntax should be executable in principle.
- Wherever a question asks you to justify a choice (e.g., why you used a particular join type, subquery vs. join, etc.), a one- or two-sentence reasoning answer is expected restating the syntax is not sufficient.
- All written answers should be your own. Do not copy definitions directly from resources.

Part I: Short Answer / Reasoning Questions

Q1. Consider a `Student` table where the `phone_number` column allows `NULL`. A classmate writes the following query to count students *without* a phone number:

```
SELECT COUNT(*) FROM Student WHERE phone_number = NULL;
```

This query always returns 0, even when such students exist. Explain why, and rewrite the query correctly. Then explain, in general terms, why `NULL` is excluded from most aggregate computations and comparison operations.

Q2. A query uses `SELECT DISTINCT department, COUNT(*) FROM Employee GROUP BY department;`. A student argues the `DISTINCT` keyword here is redundant. Is the student correct? Justify your answer by explaining what `DISTINCT` would actually be doing in this specific query.

Q3. You run two queries on the same `Customer` and `Order` tables, one using `INNER JOIN` and one using `LEFT JOIN` (Customer as the left table) and get a different row count. What does this difference in row count tell you about the data, *without looking at the tables directly*? Describe the scenario that would cause the LEFT JOIN to return more rows.

Q4. Explain why the following query is invalid in standard SQL, and correct it:

```
SELECT department, employee_name, AVG(salary)
FROM Employee
GROUP BY department;
```

In your explanation, address what it would conceptually mean for SQL to "allow" this query to run as-is i.e., what value would it even display for `employee_name`?

Q5. A manager wants the list of departments where the average salary exceeds ₹80,000. A student writes:

```
SELECT department, AVG(salary)
FROM Employee
WHERE AVG(salary) > 80000
GROUP BY department;
```

This fails. Identify the error, explain *why* `WHERE` cannot be used here (in terms of query execution order), and provide the corrected query.

Q6. For each of the following, state whether a **correlated** or **non-correlated** subquery is more appropriate, and briefly justify:

- (a) Find all products priced above the overall average product price.
- (b) Find all employees who earn more than the average salary *of their own department*.

What is the key performance implication of a correlated subquery compared to a non-correlated one when the outer table is large?

Part II: Long Answer Question

Scenario

Reuse the **online food delivery platform** schema from Assignment 1:

```
Customer(customer_id, name, email, phone)
Restaurant(restaurant_id, name, cuisine_type, rating)
MenuItem(item_id, restaurant_id, item_name, price)
Order(order_id, customer_id, restaurant_id, order_date, status)
OrderItem(order_id, item_id, quantity)
```

For each sub-question below, **write the SQL query** and **explain, in one or two sentences, why you chose the specific construct you used** (join type, subquery vs. join, presence/absence of `HAVING`, etc.).

(a) List all customers who have **never placed an order**. (Think carefully about which join type or subquery form correctly captures "never", a naive `INNER JOIN` will not work.)

(b) For each restaurant, find the **average order value**, where an order's value is the sum of `price × quantity` across its items. Only include restaurants with **more than 5 orders**.

(c) Find restaurants whose average order value (as computed in (b)) is **higher than the platform-wide average order value**. (This requires nesting your answer to (b) inside another query think about whether you need a correlated subquery here, and explain why or why not.)

(d) Find the **most ordered menu item** (by total quantity) **for each restaurant**. Briefly describe one limitation of basic `GROUP BY` + `MAX` approaches for "top-N per group" problems, even if your query handles this particular case correctly.

Part III: Normalization

Scenario

A junior developer has designed the following single table to store course enrollment data:

Enrollment(student_id, student_name, course_id, course_name,
instructor_id, instructor_name, instructor_office,
grade, enrollment_date)

Assume the following holds in this system:

- A course has exactly one instructor.
- An instructor has exactly one office.
- A student can enroll in multiple courses, and a course has multiple students.
- `(student_id, course_id)` together uniquely identify a grade and enrollment date.

Q1. List the **functional dependencies** implied by the scenario above.

Q2. Is this table in **1NF**? Justify your answer.

Q3. Identify the **partial dependency** that violates **2NF**, and explain why it is a problem in terms of update/insertion/deletion anomalies (give one concrete example of an anomaly that could occur).

Q4. Identify the **transitive dependency** that violates **3NF**, and explain the anomaly it causes.

Q5. Decompose the table into a set of relations that are in **3NF**. Use the notation `TableName(col1, col2, col3)` and underline (or otherwise indicate) each primary key. For each resulting table, briefly state which functional dependency justified separating it out.

Part IV: Design Justification (150–200 words)

Reflecting on Part II, address the following:

- For **one** query in Part II, explain a case where you could have written it using either a `JOIN` or a subquery, but chose one over the other. What factors (readability, correctness for NULL/duplicate handling, or performance) drove your choice?
- In Part III, explain why **normalizing** the `Enrollment` table trades off against **query convenience** i.e., what becomes harder to do once the table is split into 3NF form, and why might a real system sometimes intentionally keep some denormalization?